

Software Quality Assurance Documentation

Project: Elmenus Ecosystem (Mobile & Backend)

1. Test Plan & Strategy

1.1 Scope

- The testing scope covers the end-to-end integration and Regression

1.2 Testing Types

- Functional Testing: Core user journeys and feature validation.
- Integration Testing: Data flow between mobile and API.
- Regression Testing: Ensuring new features/promos don't break existing checkout flows.

1.3 Entry & Exit Criteria

- Entry: Requirements finalized, stable build in staging, test data ready.
- Exit: 100% of P0/P1 cases passed, zero critical bugs, stakeholder sign-off.

1.4 Risk Areas

- Payment Failures: Potential for double charges or order drops during gateway timeouts.
- Sync Latency: Mismatch between order placement and restaurant acceptance.

1.5 Prioritization Strategy

- P0: Critical Path (Auth, Order, Payment)
- P1: High Priority (Tracking, Search, Geolocation)
- P2: Medium/Low (Profile, Favorites, UI Polish)

2. Detailed Test Cases

ID	Scenario Type	Preconditions / Steps	Expected Result	Status
TC-01	E2E: Happy Path	1. Search "Tabali" 2. Add item 3. Select COD 4. Confirm	Order appears in "My Orders" and Merchant Dashboard instantly.	Pending
TC-02	E2E: Reorder	User has previous order. 1. Go to "My Orders" 2. Tap "Reorder"	All items added to cart with updated pricing/availability checks.	Pending
TC-03	E2E: Auth	1. Enter valid credentials 2. Tap Login	User redirected to homepage; session token stored securely.	Pending
TC-04	Edge: Net Drop	1. Click "Place Order"	No double charge. App	Pending

		2. Cut network mid-request	recovers to "Order Status" or "Retry" state.	
TC-05	Edge: Sold Out	1. Add last item 2. Wait for stock to hit 0 3. Tap Checkout	Graceful error: "Item no longer available." Cart remains intact.	Pending
TC-06	Edge: GPS Lag	1. Select restaurant near border 2. Move GPS out of range	App warns: "Restaurant too far for delivery" before payment.	Pending
TC-07	Edge: Timeout	1. Initiate payment 2. Gateway takes > 60s	User shown timeout error; order marked "Failed" not "Pending".	Pending
TC-08	Edge: Low Battery	1. Start tracking 2. Phone enters low power	App maintains tracking status without crashing background service.	Pending
TC-09	Neg: Invalid Promo	1. Apply "SAMA" to Pizza order	Error: "Code valid for Ama Sushi only." Discount not applied.	Pending
TC-10	Neg: Min Order	1. Add item < 120 EGP 2. Attempt checkout	"Checkout" button disabled; tool-tip shows remaining amount.	Pending
TC-11	Neg: Auth Bypass	1. Attempt to access /orders/123 via deep link while logged out	Redirect to login screen. No sensitive data leaked.	Pending
TC-12	Neg: Over Limit	1. Add 100 items to cart	App prevents adding more items; shows "Cart limit reached" message.	Pending
TC-13	Neg: Wrong Card	1. Use expired card for payment	Immediate rejection with clear "Card Expired" message from	Pending

			gateway.	
TC-14	Neg: SQL Injection	1. Enter "" OR 1=1" in search	System handles as literal string; no database leak/vulnerability.	Pending
TC-15	Neg: Date/Time	1. Order from "Closed" restaurant via manual system clock change	Server-side validation rejects order based on server time.	Pending

3. Endpoint 1: Fetch User Profile (Guest Session)

3.1.1 Request URL: <https://elmenus.com/2.0/profile>

Request Method: GET

Status Code: 200 OK

Expected JSON Response Payload

```
{
  "uuid": "ba8b2fa8-e085-4200-b60e-839358cc735",
  "data": {
    "shortcode": "Guest",
    "profile": null,
    "addresses": [],
    "email": "guest@elmenus.com",
    "createdAt": "2026-05-16T12:36:23.000Z",
    "hasDraft": false,
    "firstOrder": null,
    "totalOrderCount": 0,
    "canceledOrderCount": 0,
    "rejectedOrderCount": 0,
    "referredBy": null,
    "wallet": {
      "uuid": "ba8b2fa8-e085-4200-b60e-839358cc735",
      "balance": 0.0,
      "active": true
    },
    "segments": null,
    "settings": {
      "mixPanelEventLoggingEnabled": false,
```

```
    "webEngageEventLoggingEnabled": false
  }
}
}
```

3.1.2 Data Integrity & Validation Rules

- **UUID Format Validation:** The root-level uuid parameter must match standard UUIDv4 regex format.
- **Guest State Constraint Matrix:** When shortCode is exactly "Guest", business rule enforcement requires that profile is null, addresses is an empty array [], and totalOrderCount, canceledOrderCount, and rejectedOrderCount are strictly 0.
- **Strict Value Typings:** wallet.balance must be validated exclusively as a Float type to avoid rounding errors, and hasDraft must match a Boolean type.

3.2 Endpoint 2: User Authentication (Success State)

Request URL: <https://elmenus.com/api/auth-svc/1.0/auth>

Request Method: POST

Status Code: 200 OK

3.2.1 Sample Request Body Payload

```
{
  "identity": "ahmed.ali@elmenus.com",
  "password": "SecurePassword123!"
}
```

3.2.2 Expected JSON Response Payload

```
{
  "token_type": "Bearer",
  "expires_in": 3600,
```

```

“;access_token”::
“;eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1dWlkIjoieYmE4YjJmYTgtZTA4NS00MjAwLWl2MGUtODM5MzU4Y2M3MzViliwiZW1haWwiOiJhaG1lZC5hbGIAZWxtZW51cy5jb20iLCJleHAiOiE3Nzg5NzkyMDB9.signature”;;
“;refresh_token”:: “;r_77e5-4116-9f1c-5670176bce32”;;
“;user”:: {
  “;uuid”:: “;ba8b2fa8-e085-4200-b60e-839358cc735b”;;
  “;name”:: “;Ahmed Ali”;;
  “;email”:: “;ahmed.ali@elmenus.com”;;
  “;phoneNumber”:: “;+201012345678”;
}
}

```

3.2.3 Data Integrity & Validation Rules

- Cryptographic Token Contracts: access_token must return a validly structured, base64-

encoded JSON Web Token (JWT) consisting of three dot-separated sub-strings (Header.Payload.Signature).

- Lifespan & Token Definition: token_type must strictly contain the value “;Bearer”;; and

expires_in must output a positive integer (e.g., 3600), declaring session validity in seconds.

- Relational Entity Consistency: The nested object values user.uuid and user.email must map identically to the user account data referenced inside the database cluster for the authenticated profile.

4. Targeted Automation Opportunity: Mobile UI Visual Regression

Pipeline

4.1 Strategic Definition (The "What")

Implement an automated Visual Regression Testing pipeline using Robot Framework integrated with the AppiumLibrary driver backend, complemented by an intelligent computerized layout engine extension (such as AppliTools Eyes or Robot Framework WebEyes). This pipeline will specifically target dynamic, volatile discovery objects hosted on

the elmenus customer mobile viewports—including the "Only on elmenus", "Tasty Offers",

and "Hottest Deals" scrollable horizontal promotion carousels.

4.2 Architectural Engineering Justification (The "Why")

Food delivery mobile systems operate in a highly dynamic environment driven by continuous

marketing campaigns and flash promotional configurations. Traditional element-based UI test

scripts depend heavily on explicit DOM tree identifiers (such as resource IDs, XPaths, or accessibility tags). As content changes rapidly, maintaining these locators introduces significant test maintenance overhead. Migrating to an abstracted visual validation approach

delivers several key engineering advantages:

- **High Maintenance Reduction:** By validating full-screen snapshots or specific layout zones against saved, validated baseline images, scripts remain decoupled from raw element DOM mutations. Tests do not break when marketing modifies promo titles, coupon descriptions, or underlying text tags.
- **Cross-Device Layout Assurance:** Food delivery choices are highly contextual and viewport-dependent. A visual verification approach catches runtime UI defects such as text clipping on smaller viewports, text overlapping with lower navigation carousels, or

multi-line word wrapping failures that obscure pricing metrics (e.g., promotional codes like "SAMA" or delivery fees).

- Robot Framework Keyword Abstraction: Robot Framework isolates technical Appium execution scripts into clear, human-readable keywords. This enables manual engineers and product stakeholders to easily read and maintain visual validation logic without deep coding prerequisites.

5. Continuous Process Improvement: Shift-Left & Production Shadow Testing

5.1 Phase A: Pre-Development "Three Amigos" Alignment

Introduce a mandatory pre-development synchronization checkpoint including the Product

Owner, Core Developer, and Quality Assurance Architect before any product feature build

phase begins.

- Objective: Collaborate to break down requirements into explicit behavioral criteria using

standardized Given/When/Then (Gherkin) parameters before writing source code.

- System Impact: Catches structural and logical inconsistencies early—such as mapping cart clear logic rules when users alter delivery coordinates mid-browse—saving significant development time and preventing rework.

5.2 Phase B: Post-Deployment Production Shadow Testing

Establish a transparent server-side proxy service layer within production orchestration clusters to support continuous background logistics validation.

- Strategy: Intercept live, production-scale user traffic, anonymize sensitive customer metadata details, and securely duplicate/forward identical request payloads down to an isolated staging microservices environment running your testing release candidate build.

- System Impact: The dispatch matching system (assigning couriers based on geofences

and kitchen wait states) is highly volatile and difficult to accurately simulate synthetically.

Shadow testing exposes incoming release candidate logic to production-scale stress, complex traffic gridlocks, and realistic network drops without affecting live customer transactions.